FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

SISTEMAS DISTRIBUIDOS

Profesor: John Corredor, Ph.D.

TALLER DE EVALUACIÓN DE RENDIMIENTO MPI

Análisis Comparativo de Algoritmos de Multiplicación de Matrices con MPI y OpenMP en Cluster

Presentado por:

Juan Diego Ariza

Nicolas Leon

Juan Diego Pardo

Juan Sebastian Urbano

Bogotá D.C., Colombia
Marzo de 2026

Índice

1. Introducción **2**
1.1. Objetivos del Taller 2

2. Métricas de Desempeño **2**
2.1. Tiempo de Ejecución 3
2.2. Speedup 3
2.3. Eficiencia 3
2.4. Coeficiente de Variación (CV %) 3

3. Descripción de las Plataformas **3**
3.1. Configuración del Cluster 4
3.2. Consideraciones del Entorno 4

4. Diseño de Experimentos **4**
4.1. Metodología 4
4.2. Selección de Tamaños de Matriz 5
4.3. Selección de Hilos OpenMP 5
4.4. Batería de Experimentos 5
4.5. Automatización 5

5. Resultados **6**
5.1. Tabla General de Estadísticas 6
5.2. FxT vs FxC — Comparación Directa 7
5.3. Visualización de Resultados 7
5.3.1. Comparación FxT vs FxC en Serie 7
5.3.2. Efecto de Hilos OpenMP en FxT 8
5.3.3. Efecto de Hilos OpenMP en FxC 9
5.3.4. Escalabilidad MPI — FxT 9
5.3.5. Escalabilidad MPI — FxC 10
5.3.6. Speedup vs Número de Hilos 10
5.3.7. FxT vs FxC con Barras de Error 11
5.3.8. Estabilidad de las Mediciones 11

6. Análisis de Resultados **12**
6.1. FxT es consistentemente más rápido que FxC 12
6.2. OpenMP tiene poco efecto en FxT pero sí ayuda en FxC 12
6.3. El overhead de comunicación MPI limita el speedup 12
6.4. Estabilidad de las mediciones 13

7. Conclusiones y Recomendaciones **13**
7.1. Conclusiones 13
7.2. Recomendaciones 14

Referencias **15**

1. Introducción

Este informe presenta los resultados del Taller de Evaluación de Rendimiento , en el que se analizó el comportamiento de dos algoritmos de multiplicación de matrices implementados con MPI y OpenMP sobre un cluster de máquinas virtuales con sistema operativo Linux.

La multiplicación de matrices es una operación ampliamente usada en áreas como el procesamiento de imágenes, el aprendizaje automático y la simulación científica. Por su alto costo computacional, es un problema ideal para evaluar el impacto de distintas estrategias de paralelización y observar cómo se comportan los algoritmos al distribuir trabajo entre múltiples nodos y múltiples hilos dentro de cada nodo.

En este taller se trabajaron dos variantes del algoritmo:

- **mxmOmpMPIfxT (Filas x Transpuesta)**: antes de multiplicar, se calcula la transpuesta de la matriz B. Esto permite que los dos punteros que recorren el producto punto avancen secuencialmente en memoria, aprovechando mejor la caché del procesador.
- **mxmOmpMPIfxC (Filas x Columnas)**: es la multiplicación clásica, en la que se accede a B por columnas. Esto genera saltos en memoria de tamaño N en cada iteración del bucle interno, produciendo más cache misses y en general peores tiempos que FxT.

Ambos algoritmos usan MPI para distribuir el trabajo entre nodos (paralelismo de memoria distribuida) y OpenMP para paralelizar el cálculo dentro de cada nodo (paralelismo de memoria compartida). Este modelo híbrido es llamado *granularidad gruesa* (distribución entre nodos vía MPI) y *granularidad fina* (distribución entre hilos vía OpenMP).

1.1. Objetivos del Taller

1. Comparar el rendimiento de los algoritmos FxT y FxC en términos de tiempo de ejecución, speedup y coeficiente de variación.
2. Evaluar el efecto de agregar hilos OpenMP dentro de cada nodo worker.
3. Analizar el impacto de aumentar el número de workers MPI.
4. Identificar si el overhead de comunicación MPI justifica la distribución del trabajo para los tamaños de matriz evaluados.
5. Extraer recomendaciones basadas en los resultados experimentales.

2. Métricas de Desempeño

Para evaluar el rendimiento de los programas se utilizaron las siguientes métricas, que son las más comunes en la evaluación de sistemas de cómputo paralelo:

2.1. Tiempo de Ejecución

La métrica principal es el **tiempo de ejecución en microsegundos (μs)**, medido con la función `gettimeofday()` de la librería estándar de C, que ofrece precisión de microsegundos. La medición abarca únicamente la fase de cómputo y comunicación MPI, excluyendo la inicialización de matrices y la impresión de resultados.

2.2. Speedup

El speedup (S_p) cuantifica cuánto más rápido es el programa paralelo respecto a la versión serie:

$$S_p = \frac{T_1}{T_p}$$

donde T_1 es el tiempo con 1 hilo (serie) y T_p con p hilos. El speedup ideal es $S_p = p$, pero en la práctica siempre es menor debido al overhead de comunicación y sincronización.

2.3. Eficiencia

La eficiencia mide qué tan bien se aprovechan los recursos:

$$E_p = \frac{S_p}{p} \times 100\,\%$$

Una eficiencia del 100 % significaría que todos los hilos están trabajando productivamente, sin overhead.

2.4. Coeficiente de Variación (CV %)

El CV mide la estabilidad de las mediciones:

$$CV = \frac{\sigma}{\bar{x}} \times 100\,\%$$

Un CV bajo indica que las 30 repeticiones dieron tiempos similares, lo que hace el experimento confiable. Un CV alto sugiere interferencia del sistema operativo u otros factores externos.

3. Descripción de las Plataformas

Los experimentos se ejecutaron sobre un cluster de cuatro máquinas virtuales de la Universidad, todas con sistema operativo Ubuntu Linux. La Tabla 1 presenta las características de hardware de cada nodo.

Cuadro 1: Especificaciones del Cluster

Nodo	Hostname	IP	Rol	Cores	SO
Nodo 0	MIG322	10.43.100.24	MASTER	4	Ubuntu Linux
Nodo 1	MIG476	10.43.100.179	Worker	4	Ubuntu Linux
Nodo 2	MIG29	10.43.98.223	Worker	4	Ubuntu Linux
Nodo 3	MIG220	10.43.99.161	Worker	4	Ubuntu Linux

3.1. Configuración del Cluster

El cluster se configuró con las siguientes herramientas:

- **NFS (Network File System)**: el nodo MASTER exporta la carpeta `/Almacen` compartida con todos los workers, lo que permite que los ejecutables compilados estén disponibles en todos los nodos sin necesidad de copias manuales.
- **SSH sin contraseña**: el MASTER tiene acceso por llave RSA a los tres workers, requisito indispensable para que `mpirun` pueda lanzar procesos remotos automáticamente.
- **OpenMPI**: versión instalada mediante `apt` en todos los nodos (`openmpi-bin`, `openmpi-common`, `libopenmpi-dev`).

3.2. Consideraciones del Entorno

- Los experimentos se ejecutaron en un entorno de laboratorio compartido, por lo que otros procesos del sistema operativo pudieron generar cierta variabilidad en las mediciones.
- Cada configuración se ejecutó **30 veces** para obtener promedios estadísticamente robustos y reducir el impacto de valores atípicos.
- La medición de tiempo se hace en el proceso MASTER justo antes de enviar los datos a los workers y justo al recibir los últimos resultados, por lo que incluye tanto el tiempo de cómputo como el de comunicación MPI.

4. Diseño de Experimentos

4.1. Metodología

El diseño experimental evalúa todas las combinaciones de los siguientes factores:

- **Algoritmo**: FxT (Filas x Transpuesta) y FxC (Filas x Columnas)
- **Número de procesos MPI** (`-np`): 2 (1 worker) y 4 (3 workers)
- **Tamaño de matriz** (N): 512, 768, 1024, 1536 y 2048
- **Hilos OpenMP** (nH): 1, 2 y 4

4.2. Selección de Tamaños de Matriz

La Tabla 2 presenta los tamaños seleccionados con su carga computacional y memoria requerida.

Cuadro 2: Tamaños de Matrices y Carga Computacional

N	Elementos	Operaciones	Memoria aprox.	Workers válidos
512	262,144	134,217,728	2 MB	1 (512 %1=0)
768	589,824	452,984,832	4.5 MB	3 (768 %3=0)
1024	1,048,576	1,073,741,824	8 MB	1 (1024 %1=0)
1536	2,359,296	3,623,878,656	18 MB	3 (1536 %3=0)
2048	4,194,304	8,589,934,592	32 MB	1 (2048 %1=0)

Justificación: una restricción fundamental del programa es que N debe ser divisible por el número de workers, ya que el MASTER distribuye filas de forma equitativa. Por eso se eligieron tamaños compatibles con 1 o 3 workers según el caso. La progresión permite observar cómo escala el tiempo con la carga y detectar el punto donde la paralelización empieza a ser beneficiosa.

4.3. Selección de Hilos OpenMP

Se usaron 1, 2 y 4 hilos dado que cada máquina tiene 4 cores. Usar más hilos que cores disponibles generaría contención y degradaría el rendimiento, por lo que 4 es el máximo razonable.

4.4. Batería de Experimentos

La Tabla 3 presenta el resumen de todas las configuraciones ejecutadas.

Cuadro 3: Batería de Experimentos

Algoritmo	np	Workers	N	nH	Reps	N %Workers
FxT / FxC	2	1	512	1,2,4	30	0
FxT / FxC	2	1	1024	1,2,4	30	0
FxT / FxC	2	1	2048	1,2,4	30	0
FxT / FxC	4	3	768	1,2,4	30	0
FxT / FxC	4	3	1536	1,2,4	30	0

Total de ejecuciones: $2 \times 15 \times 30 = 900$ corridas.

4.5. Automatización

Todo el proceso se automatizó con el script `lanzador.pl` en Perl, que itera sobre todas las combinaciones, ejecuta cada una 30 veces, captura el tiempo impreso por el programa (en mi-

crosegundos) y lo almacena en archivos CSV separados por algoritmo (`resultados_fxt.csv` y `resultados_fxc.csv`).

5. Resultados

5.1. Tabla General de Estadísticas

La Tabla 4 y la Tabla 5 presentan las estadísticas completas de cada configuración para FxT y FxC respectivamente. Los tiempos están en segundos.

Cuadro 4: Estadísticas completas — Algoritmo FxT

np	**N**	**nH**	**Prom (s)**	**Std (s)**	**CV %**	**Speedup**
2	512	1	0.393	0.004	1.06	1.000
2	512	2	0.393	0.004	1.07	1.000
2	512	4	0.396	0.006	1.46	0.994
2	1024	1	3.125	0.039	1.26	1.000
2	1024	2	3.097	0.036	1.15	1.009
2	1024	4	3.095	0.012	0.40	1.010
2	2048	1	25.367	0.153	0.60	1.000
2	2048	2	25.437	0.116	0.45	0.997
2	2048	4	25.050	0.053	0.21	1.013
4	768	1	0.465	0.015	3.15	—
4	768	2	0.453	0.009	1.97	—
4	768	4	0.400	0.014	3.45	—
4	1536	1	3.730	0.037	0.99	—
4	1536	2	3.416	0.112	3.29	—
4	1536	4	3.100	0.050	1.60	—

Cuadro 5: Estadísticas completas — Algoritmo FxC

np	N	nH	Prom (s)	Std (s)	CV %	Speedup
2	512	1	0.732	0.016	2.25	1.000
2	512	2	0.726	0.015	2.04	1.009
2	512	4	0.733	0.021	2.89	0.999
2	1024	1	6.154	0.177	2.88	1.000
2	1024	2	6.337	0.183	2.90	0.971
2	1024	4	6.353	0.181	2.85	0.969
2	2048	1	113.342	3.442	3.04	1.000
2	2048	2	101.862	2.889	2.84	1.113
2	2048	4	96.983	2.776	2.86	1.169
4	768	1	0.891	0.022	2.43	—
4	768	2	0.848	0.034	3.97	—
4	768	4	0.725	0.014	1.91	—
4	1536	1	14.106	0.301	2.13	—
4	1536	2	11.859	0.311	2.62	—
4	1536	4	10.361	0.180	1.73	—

5.2. FxT vs FxC — Comparación Directa

La Tabla 6 muestra cuántas veces más lento es FxC respecto a FxT en cada configuración.

Cuadro 6: Razón de tiempos FxC / FxT

np	N	nH	FxT (s)	FxC (s)	FxC/FxT
2	512	1	0.393	0.732	1.86x
2	512	4	0.396	0.733	1.85x
2	1024	1	3.125	6.154	1.97x
2	1024	4	3.095	6.353	2.05x
2	2048	1	25.367	113.342	4.47x
2	2048	4	25.050	96.983	3.87x
4	768	1	0.465	0.891	1.92x
4	768	4	0.400	0.725	1.81x
4	1536	1	3.730	14.106	3.78x
4	1536	4	3.100	10.361	3.34x

5.3. Visualización de Resultados

5.3.1. Comparación FxT vs FxC en Serie

La Figura 1 muestra cómo crece el tiempo de ejecución con N para los dos algoritmos ejecutados con 1 worker y 1 hilo.

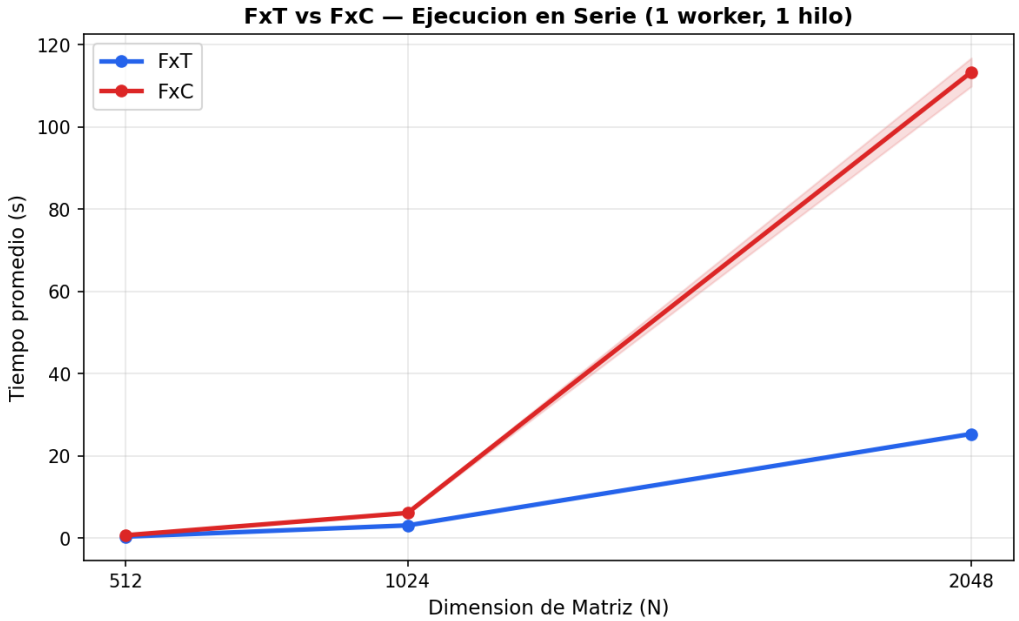

Figura 1: Tiempo de ejecución FxT vs FxC — 1 worker, 1 hilo (serie)

5.3.2. Efecto de Hilos OpenMP en FxT

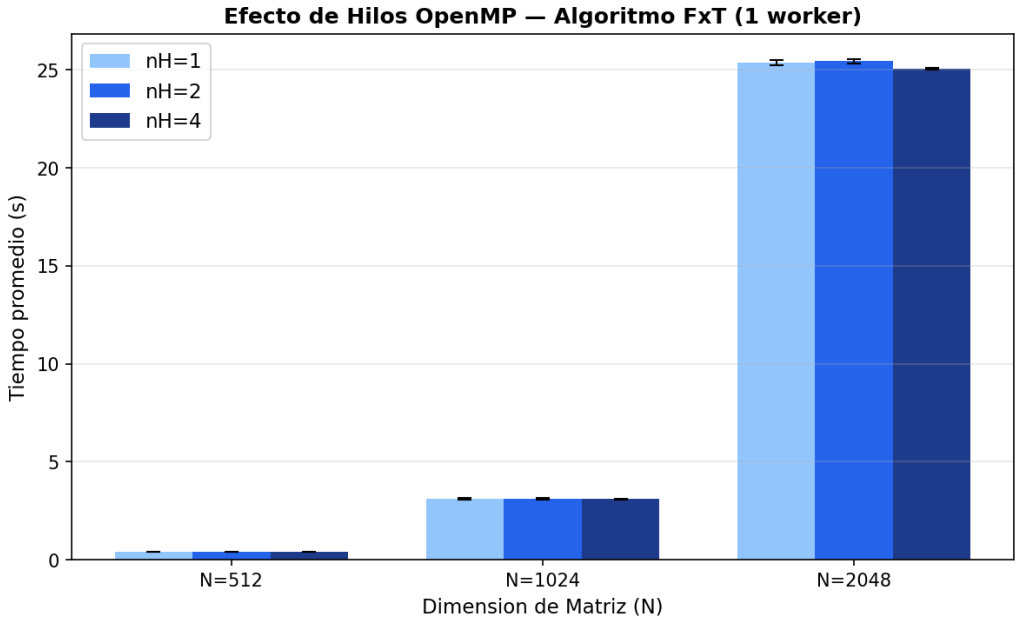

Figura 2: Efecto de hilos OpenMP en el algoritmo FxT (1 worker)

5.3.3. Efecto de Hilos OpenMP en FxC

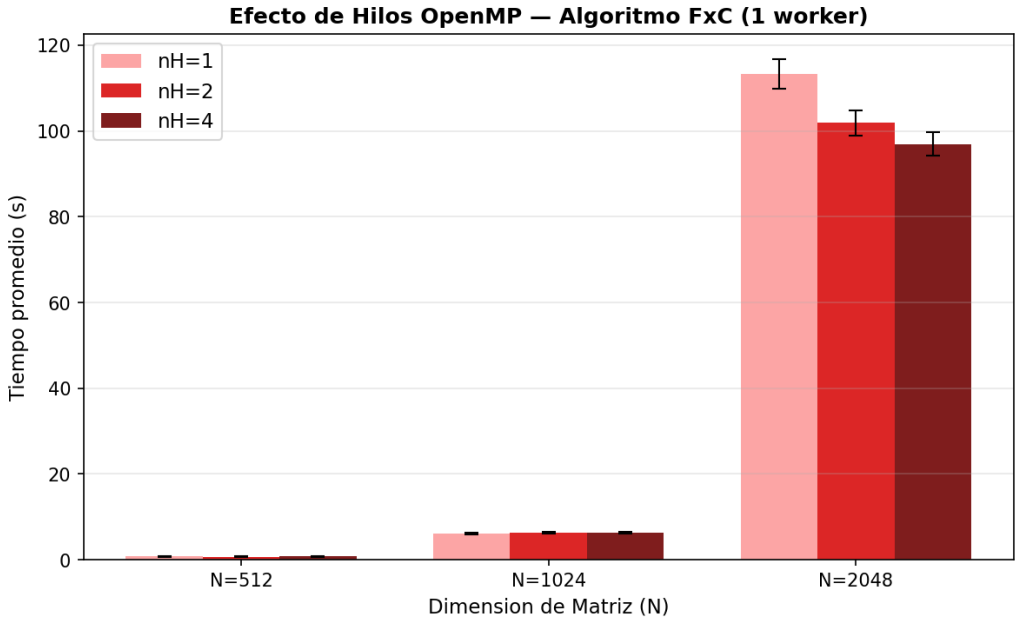

Figura 3: Efecto de hilos OpenMP en el algoritmo FxC (1 worker)

5.3.4. Escalabilidad MPI — FxT

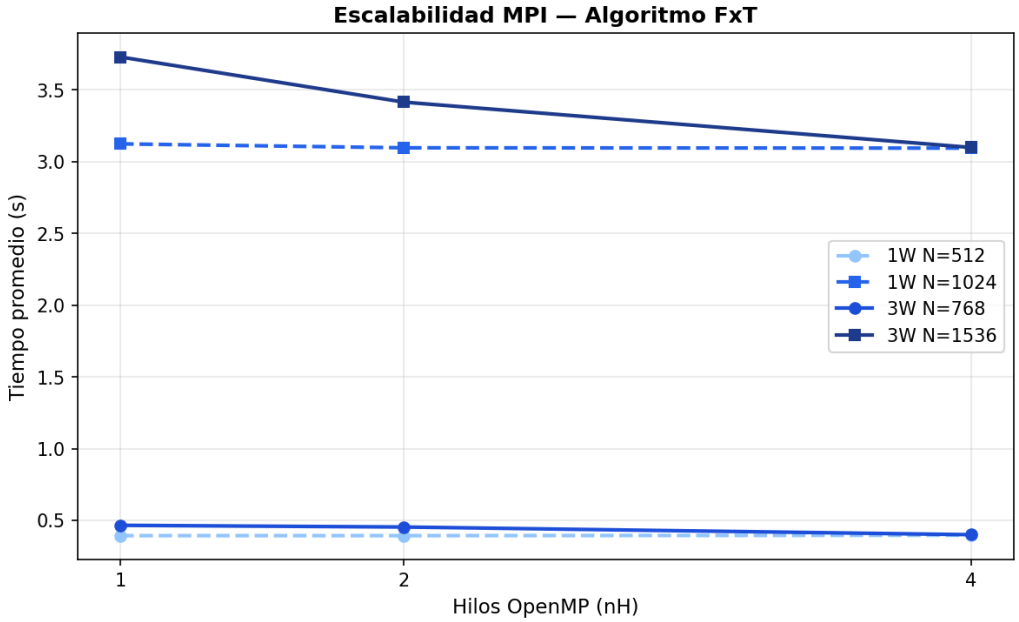

Figura 4: Escalabilidad MPI en FxT: comparación 1 worker vs 3 workers

5.3.5. Escalabilidad MPI — FxC

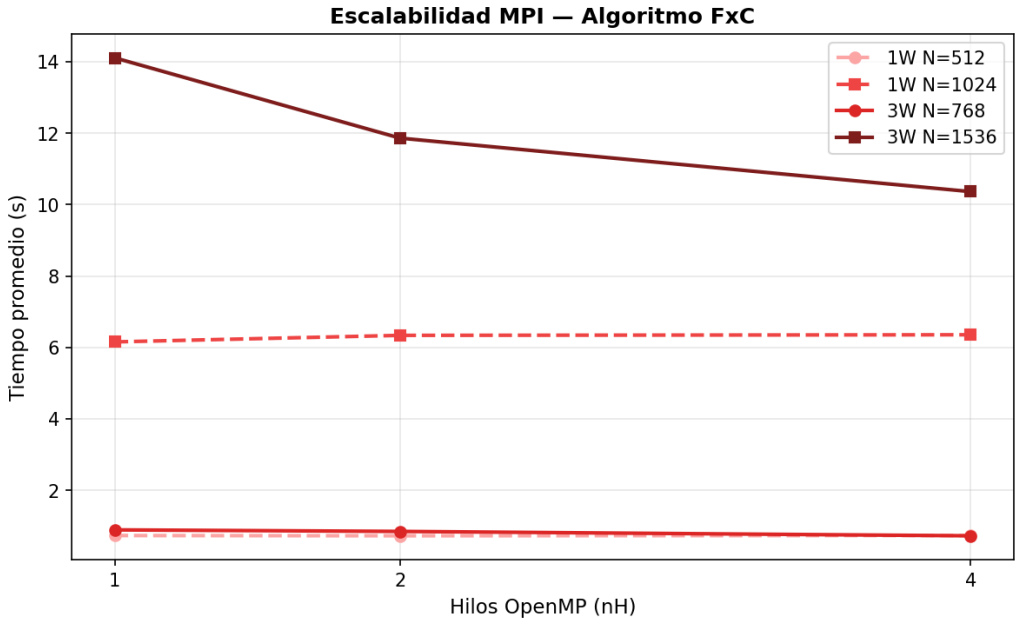

Figura 5: Escalabilidad MPI en FxC: comparación 1 worker vs 3 workers

5.3.6. Speedup vs Número de Hilos

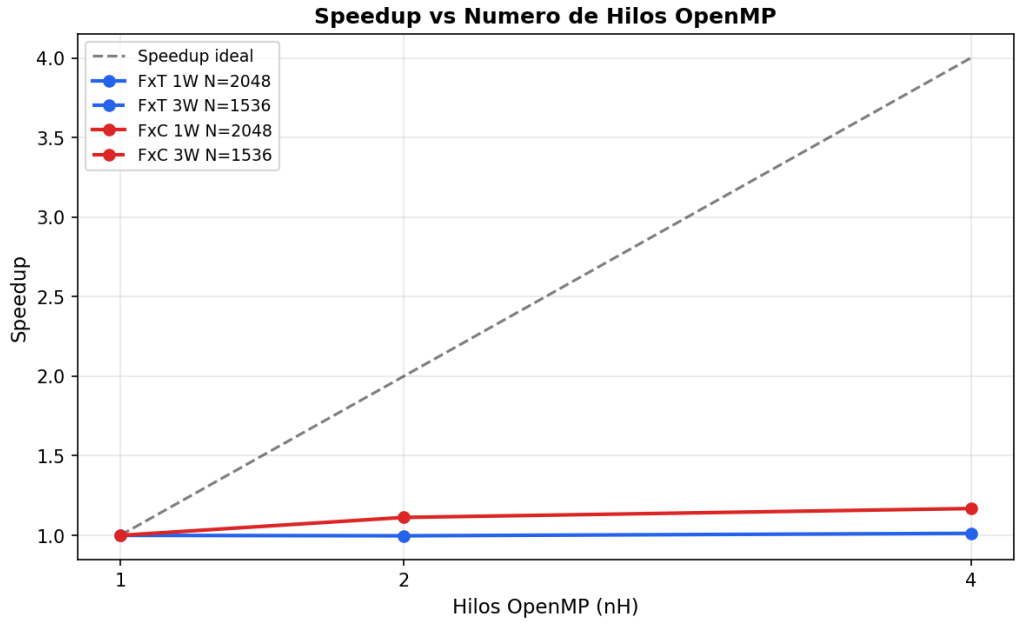

Figura 6: Speedup de FxT y FxC respecto a la ejecución serie

5.3.7. FxT vs FxC con Barras de Error

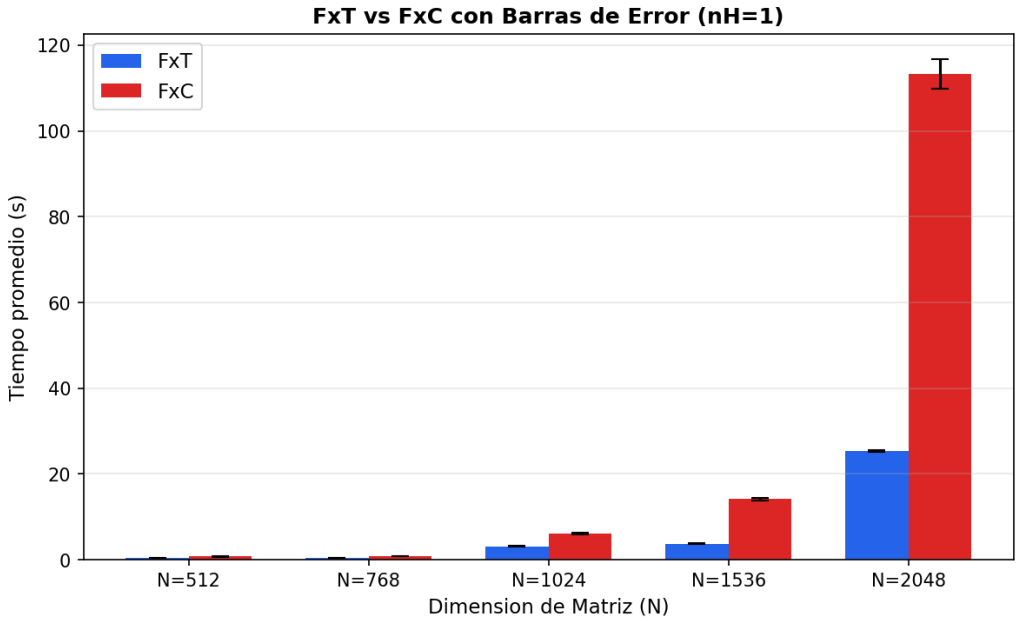

Figura 7: Comparación FxT vs FxC con barras de error (desviación estándar)

5.3.8. Estabilidad de las Mediciones

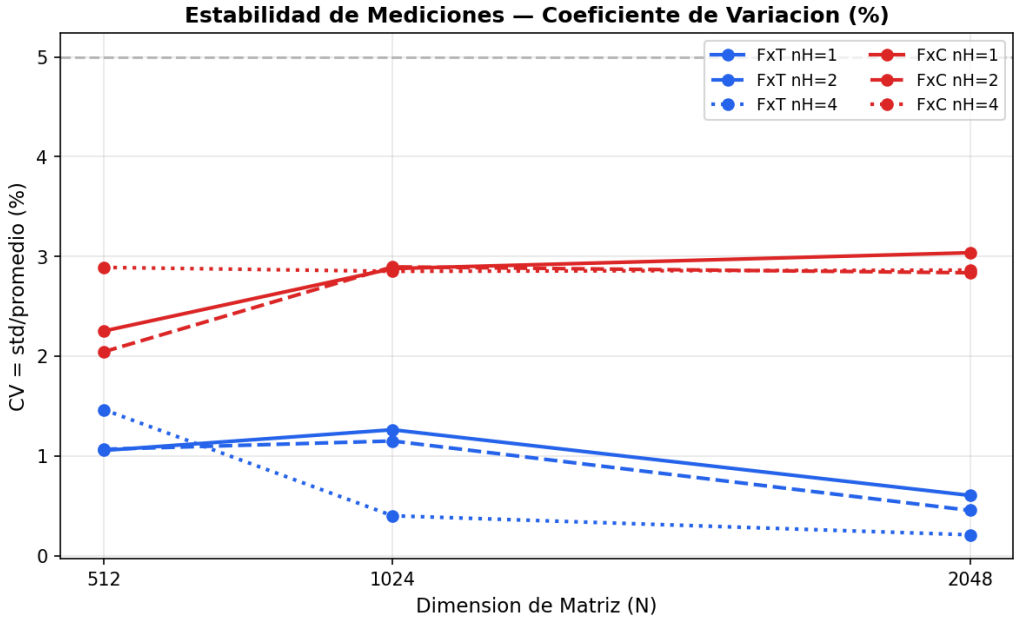

Figura 8: Coeficiente de variación (CV %) por configuración — estabilidad experimental

6. Análisis de Resultados

6.1. FxT es consistentemente más rápido que FxC

El resultado más claro del experimento es que FxT supera a FxC en todas las configuraciones evaluadas. La diferencia no es pequeña: para N=512 FxC tarda aproximadamente el doble, y para N=2048 con 1 worker llega a ser **4.47 veces más lento** (113.3 s vs 25.4 s).

La explicación está en la arquitectura de memoria del procesador. FxC accede a la matriz B por columnas: en cada iteración del bucle interno, el puntero salta N posiciones en memoria. Para N=2048, eso equivale a saltar $2048 \times 8 = 16{,}384$ bytes por acceso, lo que garantiza que el dato no esté en caché y haya que traerlo desde memoria RAM. FxT evita este problema transponiendo B antes de multiplicar, permitiendo un recorrido secuencial en ambas matrices durante el producto punto.

Lo interesante es que esta diferencia crece con N: para N=512 el ratio es 1.86x, para N=1024 es 1.97x y para N=2048 sube a 4.47x. Esto es coherente con la jerarquía de memoria: a medida que la matriz crece, deja de caber en la caché L2/L3, y los cache misses de FxC se vuelven cada vez más costosos.

6.2. OpenMP tiene poco efecto en FxT pero sí ayuda en FxC

Uno de los resultados más llamativos es que agregar hilos OpenMP prácticamente no mejora el tiempo de FxT. Para N=2048 con 1 worker, pasar de 1 a 4 hilos reduce el tiempo de 25.37 s a 25.05 s, una mejora de apenas 1.3 %. El speedup máximo obtenido por FxT fue de 1.013, básicamente nulo.

Esto no es un error sino una consecuencia directa de cómo funciona FxT: al transponer B, el producto punto ya aprovecha muy bien la caché y la mayor parte del tiempo se gasta en accesos a memoria de forma eficiente. En ese escenario, agregar hilos no ayuda mucho porque el cuello de botella ya no es la CPU sino el ancho de banda de memoria, que es un recurso compartido entre los hilos.

FxC, en cambio, sí muestra mejora con más hilos para matrices grandes: para N=2048, pasar de 1 a 4 hilos reduce el tiempo de 113.3 s a 97.0 s, una mejora del 14.4 %. Los hilos logran esconder parcialmente la latencia de los cache misses haciendo cálculos en paralelo mientras esperan los datos. Sin embargo, el speedup de 1.17x está muy lejos del ideal de 4x, lo que muestra que FxC sigue siendo fundamentalmente ineficiente.

6.3. El overhead de comunicación MPI limita el speedup

Al comparar las configuraciones con 1 worker (np=2) y 3 workers (np=4) para tamaños equivalentes, se observa que agregar workers no produce la mejora esperada. Por ejemplo:

- FxT con N=1536 y nH=4: con 3 workers tarda 3.10 s. Con 1 worker y N=1024 (carga proporcional similar) tarda 3.09 s. Es decir, usar 3 veces más nodos no redujo el tiempo proporcionalmente.
- FxT con N=768 y nH=1: con 3 workers tarda 0.465 s, mientras que con 1 worker y N=512 tarda 0.393 s.

Esto se explica por el overhead de comunicación MPI: el MASTER debe enviar la matriz B completa a todos los workers vía `MPI_Bcast` y luego recoger los resultados parciales de cada worker. Para los tamaños evaluados, este costo de comunicación consume una parte importante del tiempo total, reduciendo el beneficio de distribuir el cómputo.

Esto demuestra el concepto de *granularidad*: para que MPI valga la pena, la carga de trabajo por worker debe ser suficientemente grande como para superar el overhead de comunicación. Con matrices de hasta N=2048, el beneficio es limitado.

6.4. Estabilidad de las mediciones

Los coeficientes de variación obtenidos son en general muy bajos, lo que indica que las 30 repeticiones dieron resultados consistentes:

- FxT presenta CV entre 0.21 % y 3.45 %, siendo las mediciones más estables del experimento. Para N=2048 y nH=4 el CV fue de apenas 0.21 %, lo que es excelente.
- FxC tiene CV entre 1.73 % y 3.97 %, ligeramente más variable pero igualmente confiable.

Estos valores bajos confirman que el entorno de laboratorio fue estable durante los experimentos y que los promedios son representativos del comportamiento real de los algoritmos.

7. Conclusiones y Recomendaciones

7.1. Conclusiones

1. **FxT es superior a FxC en todos los escenarios evaluados**. La diferencia de rendimiento crece con el tamaño de la matriz, llegando a ser casi 4.5 veces más rápido para N=2048. La optimización de caché mediante la transposición de B es la causa principal de esta ventaja.
2. **OpenMP no ayuda significativamente a FxT**. Como FxT ya optimiza el acceso a memoria, el cuello de botella se desplaza al ancho de banda de memoria compartido, que no mejora con más hilos. Para FxC, los hilos sí generan una mejora visible pero lejos del speedup ideal.
3. **El overhead de comunicación MPI limita la escalabilidad**. Para los tamaños de matriz evaluados, distribuir el trabajo entre más workers no produce una reducción proporcional del tiempo, porque el costo de enviar y recibir datos entre nodos es significativo. Esto es consistente con la Ley de Amdahl: la parte secuencial (comunicación MPI) limita el speedup máximo alcanzable.
4. **Las mediciones fueron estables y confiables**. Los coeficientes de variación menores al 4 % en todas las configuraciones indican que 30 repeticiones fueron suficientes para obtener promedios representativos.
5. **El modelo híbrido MPI+OpenMP tiene potencial pero requiere mayor carga**. Para matrices más grandes (N=4096 o N=8192), el ratio cómputo/comunicación sería más favorable y se esperaría observar un beneficio real al aumentar workers y hilos.

7.2. Recomendaciones

- **Usar siempre FxT en lugar de FxC**. No tiene sentido usar FxC en producción dado que FxT es entre 2 y 4.5 veces más rápido con prácticamente la misma complejidad de implementación.
- **Para tamaños pequeños (N $\leq$ 1024), no vale la pena distribuir con MPI**. El overhead de comunicación supera el beneficio de tener más workers. En ese rango es preferible usar un solo nodo con OpenMP.
- **Escalar con matrices más grandes para aprovechar el cluster**. Si el problema lo permite, trabajar con N $\geq$ 4096 haría que el tiempo de cómputo domine sobre la comunicación y el uso de múltiples workers sí generaría speedup real.
- **Evaluar comunicación asíncrona**. Usar `MPI_Isend` y `MPI_Irecv` en lugar de las versiones bloqueantes podría reducir el tiempo que el MASTER espera ocioso mientras los workers calculan.

Referencias

1. Corredor, J. (2026). *Taller de Evaluación de Rendimiento MPI*. Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas.

2. Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann. ISBN: 978-0123742605.

3. Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press. ISBN: 978-0262533027.

4. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*, 30, 483–485.

5. OpenMP Architecture Review Board. (2024). *OpenMP Application Programming Interface Version 5.2*. Recuperado de https://www.openmp.org/specifications/

6. Open MPI Project. (2024). *Open MPI: Open Source High Performance Computing*. Recuperado de https://www.open-mpi.org/

7. Gropp, W., Lusk, E., & Skjellum, A. (1999). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (2nd ed.). MIT Press. ISBN: 978-0262571074.